



SAF

SECRETARÍA DE
ADMINISTRACIÓN
Y FINANZAS

MESA DE AYUDA

Manual de Instalación

Qué se necesita y cómo poner el sistema en producción

Contenido

1. Qué es el sistema y cómo está armado
 2. Requisitos antes de instalar
 3. Variables de entorno (archivo .env)
 4. Despliegue con Docker
 5. Conexión con el proxy externo (dominio público)
 6. Primer arranque y verificación
 7. Mantenimiento y actualizaciones
 8. Lista de verificación final
-

1 Qué es el sistema y cómo está armado

Antes de instalar conviene tener claro de qué piezas se compone el sistema y de dónde depende cada una.

Mesa de ayuda (SITickets) es un sistema de dos partes que se ejecutan por separado:

- **Backend** — API en NestJS (Node.js), expone todo bajo `/api` y genera los PDF (dictámenes, cédulas) con Puppeteer/Chromium.
- **Frontend** — aplicación Angular compilada a archivos estáticos, servidos por un contenedor nginx propio.

El sistema **no trae su propia base de datos**: se conecta a bases MySQL que ya existen y son operadas por otras áreas. En total usa **cuatro conexiones**:

Conexión	Para qué se usa	Quién la provee
Por defecto (<code>mesa_ayuda</code>)	Toda la operación propia del sistema: tickets, catálogos, usuarios, bitácora	Propia del sistema — el equipo de informática crea este esquema
<code>saf</code>	Padrón de personal del Congreso (nombre, RFC, dependencia, dirección, departamento) — es de solo lectura	Área de Recursos Humanos / Secretaría de Administración y Finanzas
<code>bienes</code>	Resguardos de equipo de cómputo (número de inventario), solo para servicios fuera de EQUIPO DE CÓMPUTO	Área de Bienes Muebles
<code>eservice</code>	Catálogo de impresoras arrendadas y nivel de tóner (lectura y escritura, la alimenta un script externo en Python)	Sistema eService (PHP/Laravel) existente

IMPORTANTE

Para EQUIPO DE CÓMPUTO, el número de inventario y el trámite de "equipo en mantenimiento"/baja **NO PASAN POR LA BASE BIENES**

, sino por una API HTTP real del sistema SIASAF (`BIENES_API_BASE_URL`). Esa API debe estar accesible desde el servidor donde corra el backend.

2 Requisitos antes de instalar

En el servidor

- Docker y Docker Compose (v2) — es la forma soportada de correr el sistema, en desarrollo y en producción.
- Acceso de red saliente, desde el servidor, hacia: el MySQL que aloja las 4 bases, y la API de SIASAF (`siasaf.gob.mx`).
- Un dominio o subruta pública detrás de un proxy (por ejemplo `https://siasaf.gob.mx/sitickets`), con HTTPS.
- Puertos libres en el host para los dos contenedores (por convención `8087` para el frontend y `3032` para el backend; son configurables).

En las bases de datos

- Un usuario de MySQL con permisos de **lectura y escritura** sobre el esquema propio del sistema (el backend crea y actualiza sus tablas solo).
- Un usuario con permiso de **solo lectura** sobre `saf` y `bienes` .
- Un usuario con lectura y escritura sobre `eservice` (ahí escribe el script de monitoreo de tóner).

NOTA SOBRE HOST.DOCKER.INTERNAL

Si el MySQL vive en la misma máquina donde corre Docker, el backend se conecta a él por `host.docker.internal` , no por `localhost` (localhost dentro del contenedor es el propio contenedor). Si el MySQL está en otro servidor de la red, se usa su IP o su nombre interno.

3 Variables de entorno

Todo lo anterior se configura en un solo archivo: `backend/.env`. Nunca se sube al repositorio — se copia a mano en el servidor a partir de la plantilla.

```
cp backend/.env.example backend/.env
# y despues se edita backend/.env con los valores reales
```

Conexión principal y comportamiento del esquema

Variable	Para qué sirve
<code>DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASS</code>	Conexión al esquema propio del sistema
<code>DB_SYNC</code>	false en producción. En true , Sequelize crea/ajusta tablas directo desde el código — solo para desarrollo.
<code>DB_MIGRAR</code>	true en producción. Aplica los <code>.sql</code> de <code>backend/sql/migraciones</code> al arrancar, uno por uno, y nunca repite uno ya aplicado.
<code>SEED_ON_BOOT / SEED_PASSWORD</code>	Si la base está vacía, siembra catálogos y el padrón de usuarios con esa contraseña inicial.

Las otras tres conexiones

Variable	Para qué sirve
<code>SAF_DB_HOST / _PORT / _NAME / _USER / _PASS</code>	Conexión de solo lectura al padrón de personal
<code>BIENES_DB_HOST / _PORT / _NAME / _USER / _PASS</code>	Conexión de solo lectura a resguardos (servicios distintos a EQUIPO DE CÓMPUTO)
<code>ESERVICE_DB_HOST / _PORT / _NAME / _USER / _PASS</code>	Conexión de lectura y escritura al catálogo de impresoras y lecturas de tóner

Integraciones externas

Variable	Para qué sirve
----------	----------------

<code>BIENES_API_BASE_URL</code>	API real de SIASAF para EQUIPO DE CÓMPUTO (consulta de resguardos, ficha técnica, alta/baja de "en mantenimiento")
<code>PYTHON_TONER_API_KEY</code>	Clave fija que debe enviar el script externo de monitoreo de tóner (header <code>X-Api-Key</code>), no usa sesión de usuario
<code>ANTHROPIC_API_KEY</code> / <code>ANTHROPIC_MODEL</code>	Solo para el botón "Mejorar redacción" del dictamen de baja. Si se deja vacío, el resto del sistema sigue funcionando; solo ese botón no responde.

Seguridad y sesión

Variable	Para qué sirve
<code>JWT_SECRET</code>	Firma los tokens de sesión. Generar uno largo y aleatorio: <code>openssl rand -base64 48</code>
<code>JWT_EXPIRES</code>	Duración de la sesión (por omisión 8h)
<code>CORS_ORIGIN</code>	Dominio exacto desde el que el navegador puede llamar a la API, por ejemplo <code>https://siasaf.gob.mx</code>
<code>CORREO_DOMINIO</code>	Dominio institucional que exige el campo "Cuenta de correo" (p. ej. <code>congresoedomex.gob.mx</code>)

ANTES DE ARRANCAR EN PRODUCCIÓN

Revisa que `DB_SYNC=false` y `DB_MIGRAR=true` . Con `DB_SYNC=true` en un servidor real, el sistema puede intentar modificar tablas directo desde los modelos antes de que corran las migraciones.

4 Despliegue con Docker

- 1 **Clonar el repositorio** en el servidor y entrar a la carpeta del proyecto.
- 2 **Crear** `backend/.env` a partir de `backend/.env.example`, con los valores reales (capítulo 3).
- 3 **Revisar** `docker-compose.prod.yml`: define dos servicios —
 - `frontend`: compila el Angular (con `BASE_HREF=/sitickets/`) y lo sirve con nginx en el puerto `8087`.
 - `backend`: compila y corre el NestJS en el puerto `3032`, monta `backend/storage` como volumen (ahí se guardan los PDF de dictamen y cédulas — **debe persistir** entre despliegues).

- 4 **Levantar los contenedores:**

```
docker compose -f docker-compose.prod.yml up -d --build
```

- 5 **Revisar el arranque del backend:**

```
docker logs sitickets-backend --tail 50
```

Debe aparecer `Migraciones: Esquema al corriente` y `Nest application successfully started`. Si es la primera vez, antes aparecerán las migraciones aplicándose una por una.

DATO UTIL

Las migraciones son automáticas e idempotentes: cada vez que se despliega una versión nueva del código, el backend revisa qué archivos de `backend/sql/migraciones` no se han aplicado todavía y solo corre esos. No hace falta correr nada a mano.

5 Conexión con el proxy externo

Los contenedores no se exponen directo a internet: quedan detrás de un servidor web existente (por ejemplo el que ya atiende `siasaf.gob.mx`), que debe reenviar dos rutas.

Ruta pública	Debe apuntar a
<code>/sitickets/</code>	Contenedor <code>frontend</code> , puerto <code>8087</code>
<code>/sitickets/backend/api/</code>	Contenedor <code>backend</code> , puerto <code>3032</code> , ruta <code>/api/</code>

Esto es así porque el frontend, cuando NO corre en `localhost:4200` (modo desarrollo), arma sus llamadas a la API con esa ruta relativa fija: `/sitickets/backend/api/`. Si el proxy externo usa otro prefijo, hay que avisar para ajustar esa constante en el código antes de compilar.

HTTPS

El certificado y el TLS los resuelve el proxy externo, no los contenedores de este sistema — ellos solo hablan HTTP puro entre sí dentro de la red del servidor.

6 Primer arranque y verificación

- 1 Con `SEED_ON_BOOT=true` y la base vacía, el sistema siembra los catálogos (servicios, problemas, prioridades) y el padrón de usuarios internos, todos con la contraseña de `SEED_PASSWORD`.
- 2 Entrar a `https://<dominio>/sitickets/entrar` y validar que cargue la pantalla de inicio de sesión.
- 3 Iniciar sesión con un RFC sembrado como **administrador** y revisar que el menú lateral muestre todas las opciones de administración (Catálogo de servicios, Catálogo de problemas, Prioridades, Registrar usuario, Reporte de tickets, Equipos dados de baja).
- 4 Registrar un ticket de prueba y confirmar que aparece en **Todos los tickets** y que el reloj y la asignación automática funcionan.
- 5 Confirmar que `backend/storage` es un volumen persistente: generar un dictamen de prueba y verificar que el archivo sobrevive a un `docker compose restart`.

7 Mantenimiento y actualizaciones

Actualizar a una versión nueva

```
git pull origin main
docker compose -f docker-compose.prod.yml up -d --build
```

Las migraciones nuevas que traiga el código se aplican solas al reiniciar el backend.

Ver logs

```
docker logs sitickets-backend --tail 100 -f
docker logs sitickets-frontend --tail 100 -f
```

RespalDOS

- La base de datos propia del sistema se respalda igual que cualquier base MySQL (`mysqldump` programado).
- La carpeta `backend/storage` (dictámenes y cédulas en PDF) debe respaldarse aparte: no vive en la base de datos.

8 Lista de verificación final

- ☐ `backend/.env` creado, con `DB_SYNC=false` y `DB_MIGRAR=true`
- ☐ `JWT_SECRET` generado de forma aleatoria (no el de ejemplo)
- ☐ Las 4 conexiones a bases de datos probadas desde el servidor
- ☐ Acceso de red confirmado hacia la API de SIASAF (`BIENES_API_BASE_URL`)
- ☐ `CORS_ORIGIN` apuntando al dominio público real
- ☐ Proxy externo reenviando `/sitickets/` y `/sitickets/backend/api/`
- ☐ Volumen de `backend/storage` persistente y con respaldo
- ☐ Login de prueba exitoso y ticket de prueba registrado correctamente